

LDSFL-Meander

A Python Code for a Reduced Two-Dimensional Morphodynamic Model
for Meandering Rivers

User Manual

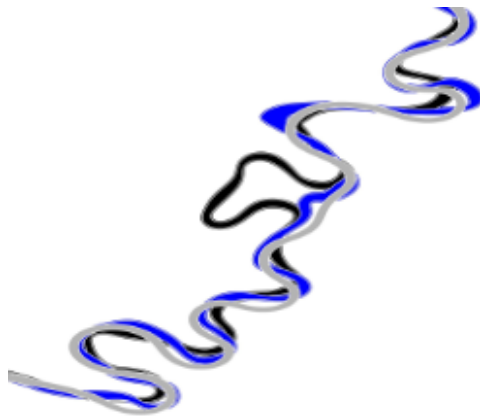
Sergio Lopez-Dubon*, Alessandro Sgarabotto, Alessandro Frascati and Stefano Lanzoni†

*Sergio.LDubon@ed.ac.uk

†stefano.lanzoni@unipd.it

What this manual covers

Installation, repository structure, command-line and GUI workflows, notation policy, dimensional and dimensionless inputs, geometry preprocessing, runtime controls, saved outputs, troubleshooting, and optional executable packaging for the current LDSFL-Meander source package.



Updated manual for LDSFL-Meander version 0.6.3

May 1, 2026

Contents

1	Introduction	3
2	Notation policy and terminology	3
3	Repository overview	3
4	Installation and first check	4
5	Quick start	4
5.1	Command line	4
5.2	GUI	4
6	Input files	5
6.1	Parameter table: <code>Input/Parameter.csv</code>	5
6.2	Geometry file: <code>Input/xy.csv</code>	6
7	Dimensionless and dimensional inputs	6
7.1	Dimensionless mode	6
7.2	Dimensional mode	7
7.3	Bed shear stress input methods	7
8	Friction options	7
8.1	Conversion formulas used by the GUI	7
9	Geometry preprocessing	8
9.1	Geometry scaling modes	8
9.2	Advanced geometry controls	8
9.3	Why scaling and filtering matter	9
10	Flexible stop criteria	9
10.1	Combining stopping criteria	10
11	Advanced solver controls	10
11.1	Boundary condition	10
11.2	Backend	10
11.3	Additional controls	10
11.4	Recommended default settings	10
12	Plot view, notifications, and saved outputs	11
12.1	Optional saved outputs	12
12.2	Completion popup	12
13	Output folders	13
14	Glossary of variables and controls	13
14.1	Reference inputs and reduced parameters	13
14.2	Geometry and future field notation	14

15 Troubleshooting	15
15.1 Geometry validation errors	15
15.2 Very slow runs	15
15.3 Unstable or suspicious results	15
16 Optional GUI executable	15
17 How to cite	16
18 License	16
19 Summary	16

1 Introduction

LDSFL-Meander is named after **Lopez-Dubon, Sgarabotto, Frascati and Lanzoni**. It is a reduced, physics-based, meander-morphodynamics model implemented in Python, with both a command-line runner and a desktop GUI. The theoretical background is described in [Frascati and Lanzoni \(2009\)](#). For more recent developments, not included in this reduced public release, see [Frascati and Lanzoni \(2013\)](#), [Bogoni et al. \(2017\)](#), and [Lopez-Dubon and Lanzoni \(2019\)](#).

LDSFL-Meander is intended for efficient reduced-model studies of channel planform evolution. It is most appropriate for wide, mildly curved, long bends and for reproducible workflow-oriented studies where the user needs to configure cases, run simulations, and inspect planform outputs. It is not intended to replace a fully-fledged 2D or 3D hydrodynamic solver, and fails to address the behaviour of sharp bends.

Core idea. The solver operates internally in terms of dimensionless quantities. The GUI can accept either dimensionless inputs directly or dimensional hydraulic/sediment inputs that are then converted into the proper internal set of dimensionless parameter before writing the solver input files.

2 Notation policy and terminology

Since the project is expected to grow toward more advanced versions, this manual distinguishes between *reference input variables* concerning the initial planform configuration, (hereafter denoted by a 0 subscript), and *flow field variables*.

Notation policy used in this manual

- Software and GUI notation uses user-oriented names and reference quantities. For example, the GUI input reference depth is denoted as D_0
- The theoretical notation uses intrinsic coordinates (s, n, z) and flow field variables such as $U(s, n)$, $V(s, n)$, $h(s, n)$, and $D(s, n)$.
- To avoid ambiguity, this manual indicate the curvature distribution along the channel axis as $\kappa(s)$.
- In this manual, B_0 denotes the **reference channel half-width**. The full width is therefore $2B_0$.

The complete list of symbols is reported in Section [14.1](#).

3 Repository overview

Path	Purpose
ldsfl/	Main Python package containing the solver, input handling, GUI utilities, outputs, flow-field routines, resistance closures, and geometry processing
run_ldsfl.py	Main command-line entry point. Use this when the input CSV files are already prepared
gui_ldsfl.py	Desktop GUI for setting up and running cases without editing the CSVs manually

Path	Purpose
Run.py	Convenience local runner for quick local execution
Reg.py	Short regression/smoke runner
Input/Parameter.csv	Case table containing the dimensionless solver inputs
Input/xy.csv	Input file for the channel axis initial geometry
smoke_test.py	Basic solver smoke test
gui_smoke_test.py	GUI/configuration/conversion smoke test
README.md	Project overview.
USER_MANUAL.md	Short Markdown guide
docs/LDSFL_Meander_user_manual.tex	Full LaTeX user manual source
docs/LDSFL_Meander_user_manual.pdf	Compiled PDF user manual
BUILD_EXE.md	Notes for building an optional packaged GUI executable
CITATION.cff	Citation metadata for the software release

4 Installation and first check

Install the dependencies from the repository root:

```
pip install -r requirements.txt
```

Then verify the source package:

```
python smoke_test.py
python gui_smoke_test.py
```

These tests confirm that the solver runs a short case and that the GUI conversion and configuration path works as expected.

5 Quick start

5.1 Command line

Use the command line (CLI) when the input files are already prepared:

```
python run_ldsfl.py --base-dir . --cases 1 --max-steps 50 --no-plots
```

The CLI also supports advanced options such as `-max-sim-time`, `-stop-mode`, `-cstab`, geometry filtering controls, and `-output-units`.

5.2 GUI

Launch the GUI with:

```
python gui_ldsfl.py
```

The GUI is the easiest way to:

- choose between dimensionless and dimensional inputs,
- validate the channel axis file,

- scale and filter geometry before writing `Input/xy.csv`,
- choose stop criteria and solver backends,
- save and load investigated configurations,
- inspect dimensionless converted parameters before the run,
- monitor live snapshots and review the final planform overlay.

An **investigated configuration** means one row in `Input/Parameter.csv`, i.e. one simulation setup with a single set of input parameters.

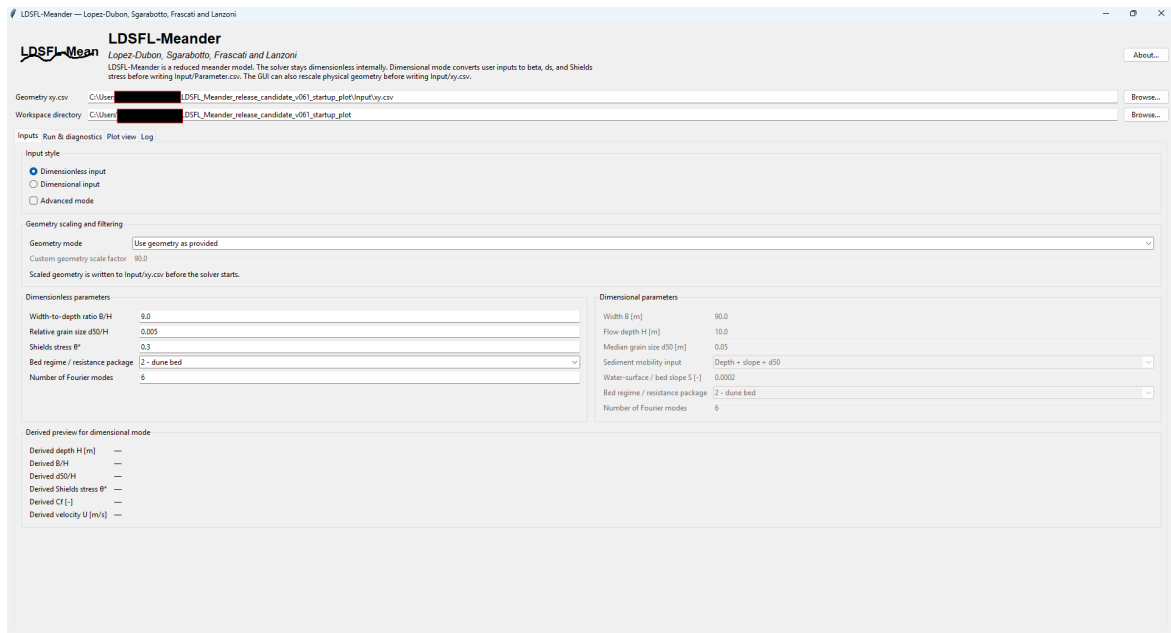


Figure 1: Main GUI view at startup. The interface preloads a bundled example of planform geometry and parameter set (*case study*), such that a first test run can be launched immediately.

At startup, the GUI loads a default example (*case study*) of initial planform configuration and input data from the repository. The planform geometry, workspace, and first-row configuration values are already defined, and the example can be run after a quick validation check.

6 Input files

6.1 Parameter table: `Input/Parameter.csv`

The code reads one row per case study. The corresponding columns are shown below.

Column	Meaning	Units
Id	Case study identifier used to select the pertinent row	-
Beta	Aspect ratio (B/D_0 , with B the channel half-width)	-
ds	Relative grain size (d_{50}/D_0)	-
Theta	Shields stress, written with the historical CSV spelling used by the code	-
flagbed	Bed-configuration flag. In the current solver: 1 = plane bed, 2 = dune covered bed	-
r	Coefficient accounting fro transverse bed-slope effects on bedload	-
Mdat	Number of Fourier modes retained in the reduced solution	-

Important notation note. In this manual, B_0 is the channel **reference half-width**, consistent with the underlying theory. The symbol **Thetha** (or **Theta** in some code paths) is retained in the CSV file to denote the dimensionless reference Shields stress, which is written in this manual as θ_0 .

6.2 Geometry file: Input/xy.csv

The geometry file stores the initial channel axis coordinates x and y as two columns. The GUI parser accepts:

- a first-row header such as x, y ,
- blank lines,
- comment lines beginning with #,
- extra columns, in which case only the first two are used.

Malformed rows, rows with fewer than two columns, or nonnumeric entries are reported with line numbers.

7 Dimensionless and dimensional inputs

7.1 Dimensionless mode

Dimensionless mode is the preferred option, especially when the parameter space needs to be explored. The main reduced-model parameters are:

- aspect ratio $\beta_0 = B_0/D_0$, with B_0 the channel half-width,
- relative grain size $d_{s0} = d_{50}/D_0$,
- Shields stress θ_0 ,
- flag specifying the bed configuration **flagbed**,
- number of Fourier modes M (stored as **Mdat**),
- coefficient accounting for transverse bed-slope effects on bedload r (available when advanced mode is activated).

7.2 Dimensional mode

Dimensional mode allows the GUI to derive the dimensionless reduced-model parameters from physical variables. Specifically, it computes the initial values of β_0 , d_{s0} , and θ_0 before writing `Input/Parameter.csv`.

7.3 Bed shear stress input methods

The GUI supports five input methods for specifying the bed shear stress associated to the initial river configuration:

1. specification of Shields stress τ_{*0}
2. specification of bed shear stress τ_{b0} and d_{50} ,
3. specification of reference depth D_0 , longitudinal slope S_0 , and d_{50} ,
4. specification of reference depth D_0 , velocity U_0 , friction, and d_{50} ,
5. specification of discharge Q , half-width B_0 , longitudinal slope S_0 , friction, and d_{50} .

8 Friction options

The GUI accepts four friction representations:

- friction coefficient c_{f0} (-),
- Darcy-Weisbach coefficient f_0 (-),
- Chézy coefficient C_{C0} ($\text{m}^{1/2}/\text{s}$),
- Manning coefficient n_{M0} ($\text{s}/\text{m}^{1/3}$).

8.1 Conversion formulas used by the GUI

The GUI uses the following conversion relations, depending on the options reported in 7.3 and 8:

$$\beta_0 = \frac{B_0}{D_0}, \quad (1a)$$

$$d_{s0} = \frac{d_{50}}{D_0}, \quad (1b)$$

$$\theta_0 = \frac{\tau_{b0}}{(\rho_s - \rho) g d_{50}}, \quad (\text{method 7.3.2}), \quad (1c)$$

$$\theta_0 = \frac{\rho D_0 S_0}{(\rho_s - \rho) d_{50}}, \quad (\text{method 7.3.3}), \quad (1d)$$

$$\theta_0 = \frac{\rho c_{f0} U_0^2}{(\rho_s - \rho) g d_{50}}, \quad (\text{method 7.3.4}), \quad (1e)$$

$$c_{f0} = \frac{f}{8}, \quad (\text{method 8.2}), \quad (1f)$$

$$c_{f0} = \frac{g}{C_{C0}^2}, \quad (\text{method 8.3}), \quad (1g)$$

$$c_{f0} = \frac{g n_{M0}^2}{D_0^{1/3}}, \quad (\text{method 8.4}), \quad (1h)$$

$$D_0 = \left(\frac{Q n_{M0}}{2 B \sqrt{S_0}} \right)^{3/5}, \quad (\text{method 7.3.5 with Manning}), \quad (1i)$$

$$D_0 = \left(\frac{Q}{2 B \sqrt{g S_0 / c_{f0}}} \right)^{2/3}, \quad (\text{method 7.3.5 with friction coefficient}), \quad (1j)$$

For the 7.3.5 method, the GUI estimates the reference depth D_0 using either the initial Manning or friction coefficient before computing θ_0 .

9 Geometry preprocessing

9.1 Geometry scaling modes

The GUI can write the imported channel axis geometry in two ways:

- use geometry as provided,
- scale x and y by the half-width B_0 .

Note that the solver entails the channel axis geometry normalised by the reference channel half-width B_0 .

9.2 Advanced geometry controls

Advanced mode includes geometry filtering controls:

- **Enable geometry smoothing/filtering** – turn the smoothing stage on/off.
- **Geometry smoothing factor** – controls how strongly short-scale noise is damped.
- **Neck cutoff interval** – check for neck cutoffs every N iterations. Set this to 0 to disable neck-cutoff checks.
- **Resample upper factor** – trigger resampling when mean point spacing becomes too large.
- **Resample lower factor** – trigger resampling when mean point spacing becomes too small.

These parameters affect curvature estimation and cutoff behavior. Use the default values first, and only tune them when you have specific numerical or scientific reasons.

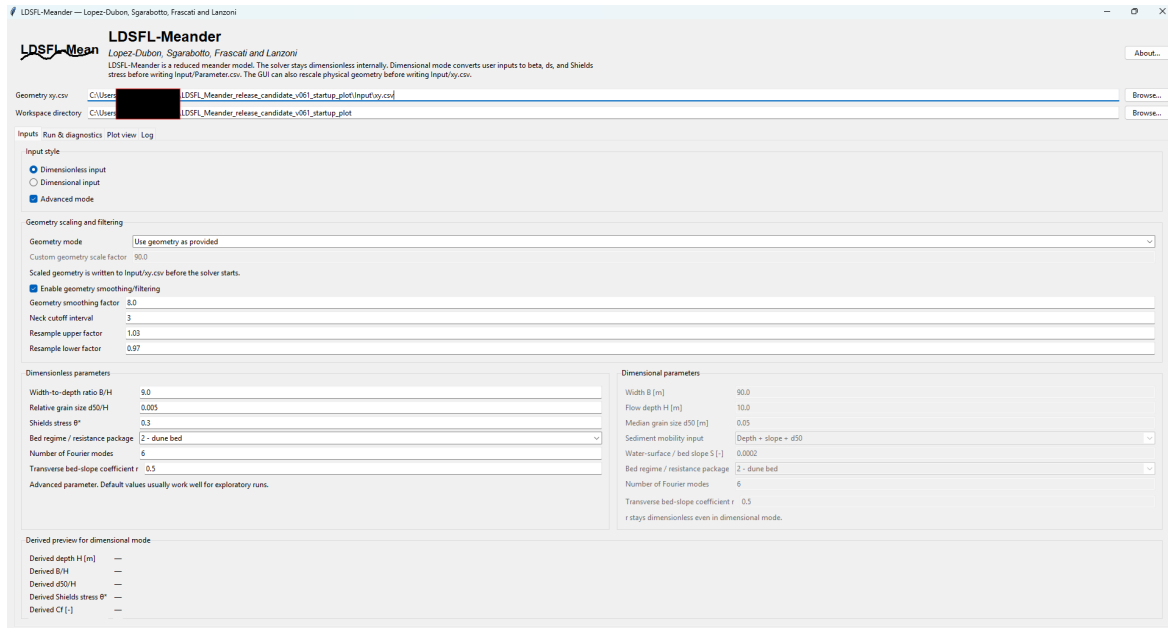


Figure 2: Example of advanced input view enabling geometry scaling/filtering controls and changing the parameter r accounting for transverse bed slope effects on bedload. These controls should normally be changed only when there are clear numerical or scientific reasons.

9.3 Why scaling and filtering matter

The reduced solver is sensitive to curvature, spacing of channel axis discretization, and geometric noise. In practice, a useful workflow is:

1. validate the imported channel axis description,
2. rescale it if needed,
3. apply mild smoothing if the source geometry is noisy,
4. keep the default resampling factors unless you have good reasons to change them.

10 Flexible stop criteria

Both the GUI and CLI support three criteria for stopping simulations:

- **steps:** maximum number of steps,
- **time:** maximum simulated time,
- **cutoffs:** maximum number of cutoffs.

Each criterion can be enabled or disabled. If only one criterion is enabled, the run behaves as a single-criterion stop.

10.1 Combining stopping criteria

Two combination rules are available:

- **first** – stop the simulation as soon as the first enabled criterion is reached,
- **all** – stop the simulation only when all enabled criteria have been reached.

For most users, the safest setup is **steps** and **cutoffs**, leaving **time** disabled initially, and use **first** as combination rule.

11 Advanced solver controls

The GUI includes an **Advanced solver controls** panel.

11.1 Boundary condition

Following [Lanzoni and Seminara \(2006\)](#), the following boundary conditions can be selected:

- **free** – open-boundary/free-flow configuration,
- **periodic** – repeating-domain/periodic-flow configuration.

11.2 Backend

- **numpy** – reference implementation,
- **numba** – JIT-accelerated implementation for supported free-boundary calculations.

11.3 Additional controls

- **cstab** – timestep stability coefficient.
- **Parallel flow flag** – low-level parallel mode switch.
- **Flow workers** – worker count for the parallel-flow option.
- **Numba parallel** – enable Numba parallel kernels where supported.
- **Numba fastmath** – enable Numba fast-math optimisations.

11.4 Recommended default settings

- start with **free + numpy**,
- then try **free + numba** if Numba is installed and you want faster runs,
- keep **cstab = 0.01** initially,
- leave **Parallel flow flag = 0** unless you are testing that path.

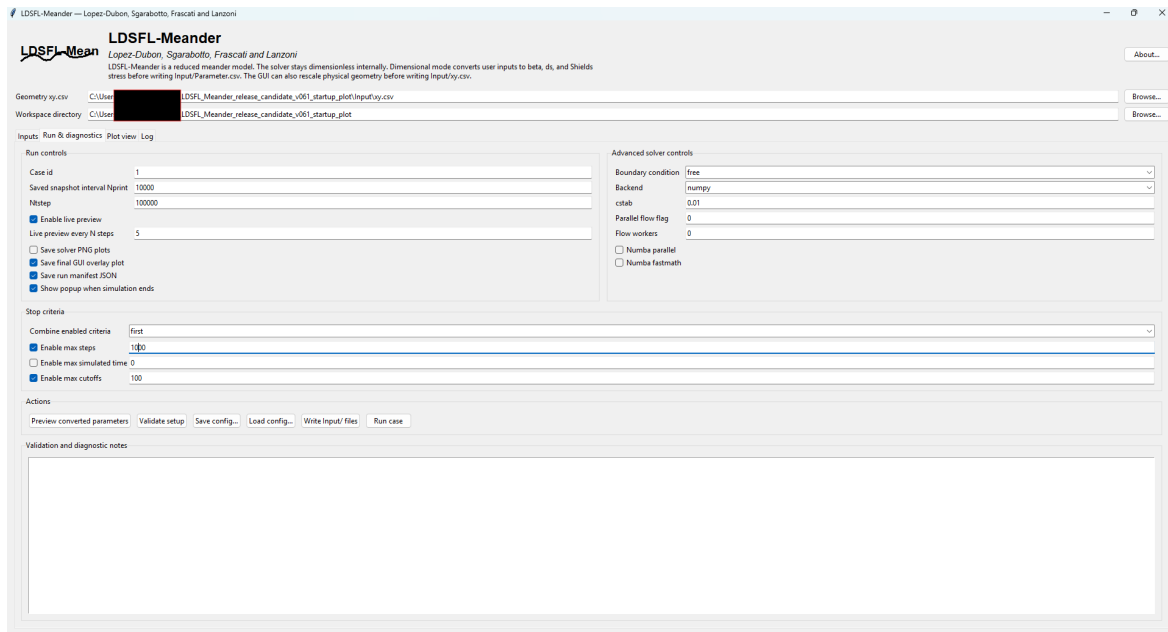


Figure 3: Example of run and diagnostics panel, including runtime controls, advanced solver options, output-unit selection, optional saved outputs, flexible stop criteria, and actions such as validation, config save/load, and case study execution.

12 Plot view, notifications, and saved outputs

The GUI plot tab shows:

- the initial channel axis geometry,
- the latest live snapshot when available,
- the final planform after the simulation finishes.

The user can also choose whether the initial channel axis geometry should remain visible on the overlay plot.

The plot title is intentionally kept at the top of the panel, while the legend is placed at the upper-left corner of the plotting area such that the curve comparison remains easy to read even for long, low-amplitude planforms.

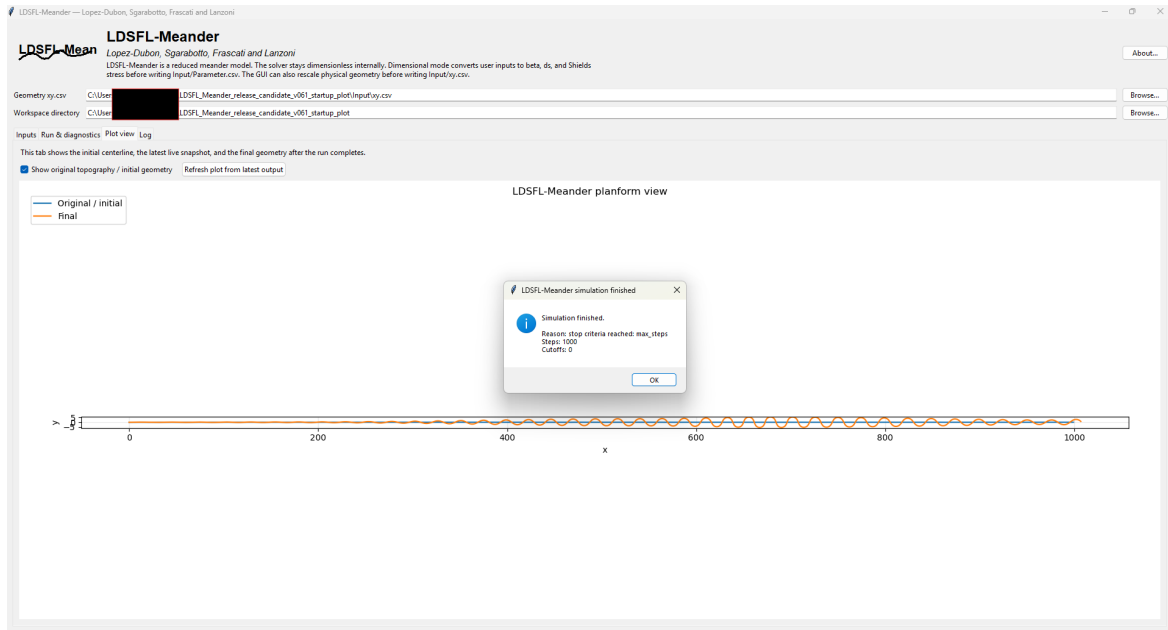


Figure 4: Example of plot view at the end of a simulation. The original and final channel centreline are overlaid, the legend is placed at the upper-left of the plotting area, and a completion popup reports the stopping reason and summary values.

12.1 Optional saved outputs

The GUI can also save:

- `gui_final_overlay.png` – the final GUI overlay plot,
- `run_manifest.json` – a machine-readable summary of the resolved inputs and simulation result.

The run controls also include an **Output units** selector:

- **dimensionless** keeps the saved solver outputs in the internal reduced variables used by the code,
- **dimensional** rescales the saved coordinates and velocity field back to dimensional form when dimensional input mode is used. Lengths are rescaled with the same geometry scale selected in the GUI (typically B_0 for horizontal coordinates and D_0 for vertical quantities), while velocity components rescaled with the resolved reference velocity U_0 .

If dimensional outputs are requested while only dimensionless inputs are provided, the code falls back to dimensionless outputs.

12.2 Completion popup

The GUI can optionally display a popup when a simulation:

- finishes successfully,
- stops because the stopping criteria were met,
- fails with an error.

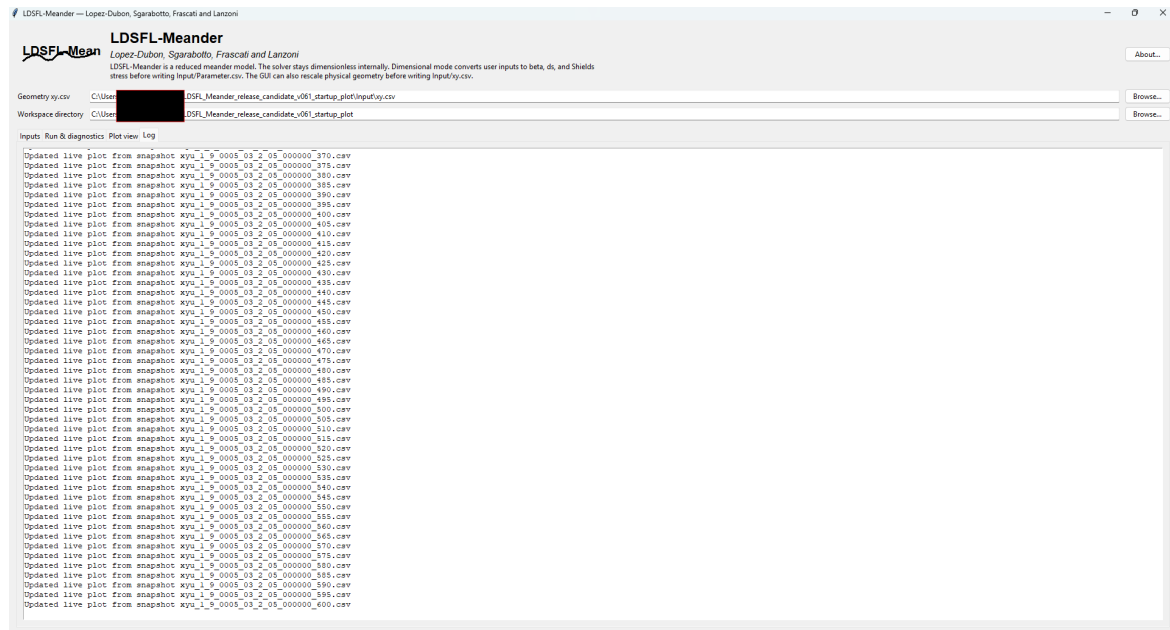


Figure 5: Example of Log tab during a simulation. The log tab records snapshot updates, status messages, and diagnostics, and is useful for verifying that live preview and saved outputs are progressing as expected.

13 Output folders

For each case, LDSFL-Meander creates:

```
Output/
  <id_files>/
    files/
    plot/
    xy_cut/
    xyu/
```

Main outputs include:

- **xyu/** – channel-axis and velocity tables through time,
- **files/** – tables of saved run variables,
- **plot/** – planform figures when solver plotting is enabled,
- **xy_cut/** – cutoff segments,
- **run_manifest.json** – resolved inputs and run summary if enabled,
- **gui_final_overlay.png** – final GUI overlay plot if enabled.

14 Glossary of variables and controls

14.1 Reference inputs and reduced parameters

Symbol / name	Meaning	Units
<i>Reference flow variables entered by the user in dimensionless mode</i>		
β_0	Aspect ratio, B_0/D_0	-
d_{s0}	Relative grain size, d_{50}/D_0	-
θ_0	Shields stress	-
flagbed	Bed-regime flag: 1 = flat bed; 2 = dune covered bed	-
r	Coefficient controlling transverse slope effects on bed-load	-
M / Mdat	Number of Fourier modes retained in the reduced solution	-
cstab	Timestep stability coefficient	-
<i>Reference flow variables entered by the user in dimensional mode</i>		
B	Channel half-width. The full width is $2B$	m
D_0	Flow depth	m
d_{50}	Median sediment grain size	m
S_0	Reach-averaged slope	-
Q	Water discharge, used in the discharge-based dimensionless conversion method	m ³ /s
U_0	Mean flow velocity, used in the velocity-based dimensionless conversion method	m/s
τ_{b0}	Bed shear stress used in the bed shear stress-based dimensionless conversion method	Pa = N/m ²
f_0	Darcy-Weisbach friction factor, used to compute the friction coefficient as $c_{f0} = f_0/8$	-
C_{C0}	Chézy coefficient, used to compute the friction coefficient as $c_{f0} = g/C_{C0}^2$.	m ^{1/2} /s
n_M	Manning coefficient; used to compute the friction coefficient as $c_f = g n_M^2 / D_0^{1/3}$	s/m ^{1/3}
<i>Variables computed for generating the dimensionless input table</i>		
c_{f0}	Friction coefficient computed from the user-supplied friction parameter and used internally by the solver.	-

14.2 Geometry and future field notation

Symbol	Meaning	SI units
x, y	Cartesian planform coordinates stored in <code>Input/xy.csv</code> . When geometry is scaled, these may become normalized coordinates	(m or -)
s	Intrinsic longitudinal coordinate along the channel axis	(m or -)
n	Intrinsic transverse coordinate	(m or -)
z	Vertical coordinate	(m or -)
$\kappa(s)$	Curvature distribution of the channel centreline	(m ⁻¹ or -)
$B(s)$	Channel half-width distribution (in future variable-width code extensions)	(m)
$U(s, n)$	Streamwise depth-averaged velocity	(m/s or -)
$V(s, n)$	Transverse depth-averaged velocity	(m/s or -)

Symbol	Meaning	SI units
$D(s, n)$	Local depth	(m or -)
$h(s, n)$	Free-surface elevation	(m or -)

15 Troubleshooting

15.1 Geometry validation errors

If the GUI reports malformed initial channel axis geometry:

- check for nonnumeric rows,
- check that each data row has at least two columns,
- remove duplicated consecutive points if possible,
- check that the point spacing is not significantly uneven.

15.2 Very slow runs

Try to:

- reduce the number of Fourier modes (but not less than 6),
- use the Numba backend,
- leave the parallel-flow flag at 0 unless benchmarking,
- increase `Nprint` if intermediate snapshots are too frequent.

15.3 Unstable or suspicious results

Try to:

- reduce `cstab`,
- validate the initial channel axis geometry file carefully,
- check whether the dimensional conversions produce realistic β_0 , d_{s0} , and θ_0 ,
- return to the default geometry filtering settings.

16 Optional GUI executable

The clean release workflow is:

1. keep the source repository as the main scientific release,
2. build the Windows GUI executable from a tagged source release,
3. upload the executable bundle as a GitHub release asset,
4. keep Zenodo tied to the source tag rather than the binary.

The basic build command is:

```

pip install pyinstaller
pyinstaller --noconfirm --clean --windowed --name LDSFL-Meander-GUI gui_ldsfl.
py

```


17 How to cite

Citation for the software:

Lopez Dubon, S., Sgarabotto, A., Frascati, A., Lanzoni, S. (2026). LDSFL-Meander: A python code for a reduced two-dimensional morphodynamic model for meandering rivers. [*Software*]. GitHub. doi: 10.5281/zenodo.19945291

Citations for the reduced two-dimensional meander-morphodynamics model:

Lopez Dubon, S. and Lanzoni, S. (2019). Meandering evolution and width variations: A physics-statistics-based modeling approach. *Water Resources Research*, 55(1), 76–94. doi: 10.1029/2018WR023639

Frascati, A. and Lanzoni, S. (2009). Morphodynamic regime and long-term evolution of meandering rivers. *Journal of Geophysical Research: Earth Surface*, 114(F2), 1–12. doi: 10.1029/2008JF001101

18 License

This software is released under the MIT License. See the repository LICENSE file for the full text.

19 Summary

LDSFL-Meander already provides a strong source-package workflow for reduced two-dimensional meander modelling. It has a documented command-line path, a GUI with dimensional and dimensionless input modes, explicit geometry scaling by B_0 , output-unit selection, validation, plotting, and smoke tests. It is crucial to understand the dimensional conversion features as a convenience layer that feeds the internal dimensionless solver consistently. The model is appropriate for wide, mildly curved, long bends.

For any questions, bug reports, or suggestions, please contact the developers via email at Sergio.LDubon@ed.ac.uk or stefano.lanzoni@unipd.it. We appreciate your feedback to improve the software.

References

- Bogoni, M., Putti, M., and Lanzoni, S. (2017). Modeling meander morphodynamics over self-formed heterogeneous floodplains. *Water Resources Research*, 53(6), 5137–5157. doi: 10.1002/2017WR020726.
- Frascati, A. and Lanzoni, S. (2009). Morphodynamic regime and long-term evolution of meandering rivers. *Journal of Geophysical Research: Earth Surface*, 114(F2), 1–12. doi: 10.1029/2008JF001101.
- Frascati, A. and Lanzoni, S. (2013). A mathematical model for meandering rivers with varying width. *Journal of Geophysical Research: Earth Surface*, 118, 1641–1657. doi: 10.1002/jgrf.20084.
- Lanzoni, S. and Seminara, G. (2006). On the nature of meander instability. *Journal of Geophysical Research: Earth Surface*, 111(F4), 1–14. doi: 10.1029/2005JF000416.
- Lopez-Dubon, S. and Lanzoni, S. (2019). Meandering evolution and width variations: A physics-statistics-based modeling approach. *Water Resources Research*, 55(1), 76–94. doi: 10.1029/2018WR023639.

Seminara, G., Lanzoni, S., and Tambroni, N. (2023). *Theoretical Morphodynamics: River Meandering*. Firenze University Press. doi: 10.36253/979-12-215-0303-6.

Zolezzi, G. and Seminara, G. (2001). Downstream and upstream influence in river meandering. Part 1. General theory and application to overdeepening. *Journal of Fluid Mechanics*, 438, 183–211. doi: 10.1017/S002211200100427X.